

What is the runtime of this approach? We have nested for loops, each of which runs for k iterations. The runtime of the `allPossibleKFactors` is $O(k^3)$. Then, we sort the k^3 results in $O(k^3 \log(k^3))$ time (which is equivalent to $O(k^3 \log k)$). This gives us a runtime of $O(k^3 \log k)$.

There are a number of optimizations you could make to this (and better ways of handling the integer overflow), but honestly this algorithm is fairly slow. We should instead focus on reworking the algorithm.

Improved

Let's picture what our results will look like.

1	-	$3^0 * 5^0 * 7^0$
3	3	$3^1 * 5^0 * 7^0$
5	5	$3^0 * 5^1 * 7^0$
7	7	$3^0 * 5^0 * 7^1$
9	$3*3$	$3^2 * 5^0 * 7^0$
15	$3*5$	$3^1 * 5^1 * 7^0$
21	$3*7$	$3^1 * 5^0 * 7^1$
25	$5*5$	$3^0 * 5^2 * 7^0$
27	$3*9$	$3^3 * 5^0 * 7^0$
35	$5*7$	$3^0 * 5^1 * 7^1$
45	$5*9$	$3^2 * 5^1 * 7^0$
49	$7*7$	$3^0 * 5^0 * 7^2$
63	$3*21$	$3^2 * 5^0 * 7^1$

The question is: what is the next value in the list? The next value will be one of these:

- $3 * (\text{some previous number in list})$
- $5 * (\text{some previous number in list})$
- $7 * (\text{some previous number in list})$

If this doesn't immediately jump out at you, think about it this way: whatever the next value (let's call it nv) is, divide it by 3. Will that number have already appeared? As long as nv has factors of 3 in it, yes. The same can be said for dividing it by 5 and 7.

So, we know A_k can be expressed as $(3, 5 \text{ or } 7) * (\text{some value in } \{A_1, \dots, A_{k-1}\})$. We also know that A_k is, by definition, the next number in the list. Therefore, A_k will be the smallest "new" number (a number that it's already in $\{A_1, \dots, A_{k-1}\}$) that can be formed by multiplying each value in the list by 3, 5 or 7.

How would we find A_k ? Well, we could actually multiply each number in the list by 3, 5, and 7 and find the smallest element that has not yet been added to our list. This solution is $O(k^2)$. Not bad, but I think we can do better.

Rather than A_k trying to "pull" from a previous element in the list (by multiplying all of them by 3, 5 and 7), we can think about each previous value in the list as "pushing" out three subsequent values in the list. That is, each number A_i will eventually be used later in the list in the following forms:

- $3 * A_i$
- $5 * A_i$
- $7 * A_i$

We can use this thought to plan in advance. Each time we add a number A_i to the list, we hold on to the values $3A_i$, $5A_i$, and $7A_i$ in some sort of temporary list. To generate A_{i+1} , we search through this temporary list to find the smallest value.

Our code looks like this:

```

1  int removeMin(Queue<Integer> q) {
2      int min = q.peek();
3      for (Integer v : q) {
4          if (min > v) {
5              min = v;
6          }
7      }
8      while (q.contains(min)) {
9          q.remove(min);
10     }
11     return min;
12 }

13
14 void addProducts(Queue<Integer> q, int v) {
15     q.add(v * 3);
16     q.add(v * 5);
17     q.add(v * 7);
18 }
19
20 int getKthMagicNumber(int k) {
21     if (k < 0) return 0;
22
23     int val = 1;
24     Queue<Integer> q = new LinkedList<Integer>();
25     addProducts(q, 1);
26     for (int i = 0; i < k; i++) {
27         val = removeMin(q);
28         addProducts(q, val);
29     }
30     return val;
31 }

```

This algorithm is certainly much, much better than our first algorithm, but it's still not quite perfect.

Optimal Algorithm

To generate a new element A_i , we are searching through a linked list where each element looks like one of:

- $3 \times$ previous element
- $5 \times$ previous element
- $7 \times$ previous element

Where is there unnecessary work that we might be able to optimize out?

Let's imagine our list looks like:

$$q_6 = \{7A_1, 5A_2, 7A_2, 7A_3, 3A_4, 5A_4, 7A_4, 5A_5, 7A_5\}$$

When we search this list for the min, we check if $7A_1 < \min$, and then later we check if $7A_5 < \min$. That seems sort of silly, doesn't it? Since we know that $A_1 < A_5$, we should only need to check $7A_1$.

If we separated the list from the beginning by the constant factors, then we'd only need to check the first of the multiples of 3, 5 and 7. All subsequent elements would be bigger.

That is, our list above would look like:

```
Q36 = {3A4}
Q56 = {5A2, 5A4, 5A5}
Q76 = {7A1, 7A2, 7A3, 7A4, 7A5}
```

To get the min, we only need to look at the fronts of each queue:

```
y = min(Q3.head(), Q5.head(), Q7.head())
```

Once we compute y , we need to insert $3y$ into Q3, $5y$ into Q5, and $7y$ into Q7. But, we only want to insert these elements if they aren't already in another list.

Why might, for example, $3y$ already be somewhere in the holding queues? Well, if y was pulled from Q7, then that means that $y = 7x$, for some smaller x . If $7x$ is the smallest value, we must have already seen $3x$. And what did we do when we saw $3x$? We inserted $7 * 3x$ into Q7. Note that $7 * 3x = 3 * 7x = 3y$.

To put this another way, if we pull an element from Q7, it will look like $7 * \text{suffix}$, and we know we have already handled $3 * \text{suffix}$ and $5 * \text{suffix}$. In handling $3 * \text{suffix}$, we inserted $7 * 3 * \text{suffix}$ into a Q7. And in handling $5 * \text{suffix}$, we know we inserted $7 * 5 * \text{suffix}$ in Q7. The only value we haven't seen yet is $7 * 7 * \text{suffix}$, so we just insert $7 * 7 * \text{suffix}$ into Q7.

Let's walk through this with an example to make it really clear.

```
initialize:
    Q3 = 3
    Q5 = 5
    Q7 = 7
remove min = 3. insert 3*3 in Q3, 5*3 into Q5, 7*3 into Q7.
    Q3 = 3*3
    Q5 = 5, 5*3
    Q7 = 7, 7*3
remove min = 5. 3*5 is a dup, since we already did 5*3. insert 5*5 into Q5, 7*5
into Q7.
    Q3 = 3*3
    Q5 = 5*3, 5*5
    Q7 = 7, 7*3, 7*5.
remove min = 7. 3*7 and 5*7 are dups, since we already did 7*3 and 7*5. insert 7*7
into Q7.
    Q3 = 3*3
    Q5 = 5*3, 5*5
    Q7 = 7*3, 7*5, 7*7
remove min = 3*3 = 9. insert 3*3*3 in Q3, 3*3*5 into Q5, 3*3*7 into Q7.
    Q3 = 3*3*3
    Q5 = 5*3, 5*5, 5*3*3
    Q7 = 7*3, 7*5, 7*7, 7*3*3
remove min = 5*3 = 15. 3*(5*3) is a dup, since we already did 5*(3*3). insert
5*5*3 in Q5, 7*5*3 into Q7.
    Q3 = 3*3*3
    Q5 = 5*5, 5*3*3, 5*5*3
    Q7 = 7*3, 7*5, 7*7, 7*3*3, 7*5*3
remove min = 7*3 = 21. 3*(7*3) and 5*(7*3) are dups, since we already did 7*(3*3)
and 7*(5*3). insert 7*7*3 into Q7.
    Q3 = 3*3*3
    Q5 = 5*5, 5*3*3, 5*5*3
    Q7 = 7*5, 7*7, 7*3*3, 7*5*3, 7*7*3
```

Our pseudocode for this problem is as follows:

1. Initialize array and queues Q3, Q5, and Q7

2. Insert 1 into array.
3. Insert $1*3$, $1*5$ and $1*7$ into Q3, Q5, and Q7 respectively.
4. Let x be the minimum element in Q3, Q5, and Q7. Append x to magic.
5. If x was found in:
 - Q3 -> append $x*3$, $x*5$ and $x*7$ to Q3, Q5, and Q7. Remove x from Q3.
 - Q5 -> append $x*5$ and $x*7$ to Q5 and Q7. Remove x from Q5.
 - Q7 -> only append $x*7$ to Q7. Remove x from Q7.
6. Repeat steps 4 - 6 until we've found k elements.

The code below implements this algorithm.

```

1  int getKthMagicNumber(int k) {
2      if (k < 0) {
3          return 0;
4      }
5      int val = 0;
6      Queue<Integer> queue3 = new LinkedList<Integer>();
7      Queue<Integer> queue5 = new LinkedList<Integer>();
8      Queue<Integer> queue7 = new LinkedList<Integer>();
9      queue3.add(1);
10
11     /* Include 0th through kth iteration */
12     for (int i = 0; i <= k; i++) {
13         int v3 = queue3.size() > 0 ? queue3.peek() : Integer.MAX_VALUE;
14         int v5 = queue5.size() > 0 ? queue5.peek() : Integer.MAX_VALUE;
15         int v7 = queue7.size() > 0 ? queue7.peek() : Integer.MAX_VALUE;
16         val = Math.min(v3, Math.min(v5, v7));
17         if (val == v3) { // enqueue into queue 3, 5 and 7
18             queue3.remove();
19             queue3.add(3 * val);
20             queue5.add(5 * val);
21         } else if (val == v5) { // enqueue into queue 5 and 7
22             queue5.remove();
23             queue5.add(5 * val);
24         } else if (val == v7) { // enqueue into Q7
25             queue7.remove();
26         }
27         queue7.add(7 * val); // Always enqueue into Q7
28     }
29     return val;
30 }

```

When you get this question, do your best to solve it—even though it's really difficult. You can start with a brute force approach (challenging, but not quite as tricky), and then you can start trying to optimize it. Or, try to find a pattern in the numbers.

Chances are that your interviewer will help you along when you get stuck. Whatever you do, don't give up! Think out loud, wonder out loud, and explain your thought process. Your interviewer will probably jump in to guide you.

Remember, perfection on this problem is not expected. Your performance is evaluated in comparison to other candidates. Everyone struggles on a tricky problem.

17.10 Majority Element: A majority element is an element that makes up more than half of the items in an array. Given a positive integers array, find the majority element. If there is no majority element, return -1. Do this in $O(N)$ time and $O(1)$ space.

Input: 1 2 5 9 5 9 5 5 5

Output: 5

pg 187

SOLUTION

Let's start off with an example:

3 1 7 1 3 7 3 7 1 7 7

One thing we can notice here is that if the majority element (in this case 7) appears less often in the beginning, it must appear much more often toward the end. That's a good observation to make.

This interview question specifically requires us to do this in $O(N)$ time and $O(1)$ space. Nonetheless, sometimes it can be useful to relax one of those requirements and develop an algorithm. Let's try relaxing the time requirement but staying firm on the $O(1)$ space requirement.

Solution #1 (Slow)

One simple way to do this is to just iterate through the array and check each element for whether it's the majority element. This takes $O(N^2)$ time and $O(1)$ space.

```
1 int findMajorityElement(int[] array) {
2     for (int x : array) {
3         if (validate(array, x)) {
4             return x;
5         }
6     }
7     return -1;
8 }

9
10 boolean validate(int[] array, int majority) {
11     int count = 0;
12     for (int n : array) {
13         if (n == majority) {
14             count++;
15         }
16     }
17
18     return count > array.length / 2;
19 }
```

This does not fit the time requirements of the problem, but it is potentially a starting point. We can think about optimizing this.

Solution #2 (Optimal)

Let's think about what that algorithm did on a particular example. Is there anything we can get rid of?

3	1	7	1	1	7	7	3	7	7	7
0	1	2	3	4	5	6	7	8	9	10

In the very first validation pass, we select 3 and validate it as the majority element. Several elements later, we've still counted just one 3 and several non-3 elements. Do we need to continue checking for 3?

On one hand, yes. 3 could redeem itself and be the majority element, if there are a bunch of 3s later in the array.

On the other hand, not really. If 3 does redeem itself, then we'll encounter those 3s later on, in a subsequent validation step. We could terminate this `validate(3)` step.

That logic is fine for the first element, but what about the next one? We would immediately terminate `validate(1)`, `validate(7)`, and so on.

Since the logic was okay for the first element, what if we treated all subsequent elements like they're the first element of some new subarray? This would mean that we start `validate(array[1])` at index 1, `validate(array[2])` at index 2, and so on.

What would this look like?

```
validate(3)
  sees 3 -> countYes = 1, countNo = 0
  sees 1 -> countYes = 1, countNo = 1
  TERMINATE. 3 is not majority thus far.
validate(1)
  sees 1 -> countYes = 0, countNo = 0
  sees 7 -> countYes = 1, countNo = 1
  TERMINATE. 1 is not majority thus far.
validate(7)
  sees 7 -> countYes = 1, countNo = 0
  sees 1 -> countYes = 1, countNo = 1
  TERMINATE. 7 is not majority thus far.
validate(1)
  sees 1 -> countYes = 1, countNo = 0
  sees 1 -> countYes = 2, countNo = 0
  sees 7 -> countYes = 2, countNo = 1
  sees 7 -> countYes = 2, countNo = 1
  TERMINATE. 1 is not majority thus far.
validate(1)
  sees 1 -> countYes = 1, countNo = 0
  sees 7 -> countYes = 1, countNo = 1
  TERMINATE. 1 is not majority thus far.
validate(7)
  sees 7 -> countYes = 1, countNo = 0
  sees 7 -> countYes = 2, countNo = 0
  sees 3 -> countYes = 2, countNo = 1
  sees 7 -> countYes = 3, countNo = 1
  sees 7 -> countYes = 4, countNo = 1
  sees 7 -> countYes = 5, countNo = 1
```

Do we know at this point that 7 is the majority element? Not necessarily. We have eliminated everything before that 7, and everything after it. But there could be no majority element. A quick `validate(7)` pass that starts from the beginning can confirm if 7 is actually the majority element. This `validate` step will be $O(N)$ time, which is also our Best Conceivable Runtime. Therefore, this final `validate` step won't impact our total runtime.

This is pretty good, but let's see if we can make this a bit faster. We should notice that some elements are being "inspected" repeatedly. Can we get rid of this?

Look at the first `validate(3)`. This fails after the subarray `[3, 1]`, because 3 was not the majority element. But because `validate` fails the instant an element is not the majority element, it also means nothing else in that subarray was the majority element. By our earlier logic, we don't need to call `validate(1)`. We know that 1 did not appear more than half the time. If it is the majority element, it'll pop up later.

Let's try this again and see if it works out.

```

validate(3)
  sees 3 -> countYes = 1, countNo = 0
  sees 1 -> countYes = 1, countNo = 1
  TERMINATE. 3 is not majority thus far.
skip 1
validate(7)
  sees 7 -> countYes = 1, countNo = 0
  sees 1 -> countYes = 1, countNo = 1
  TERMINATE. 7 is not majority thus far.
skip 1
validate(1)
  sees 1 -> countYes = 1, countNo = 0
  sees 7 -> countYes = 1, countNo = 1
  TERMINATE. 1 is not majority thus far.
skip 7
validate(7)
  sees 7 -> countYes = 1, countNo = 0
  sees 3 -> countYes = 1, countNo = 1
  TERMINATE. 7 is not majority thus far.
skip 3
validate(7)
  sees 7 -> countYes = 1, countNo = 0
  sees 7 -> countYes = 2, countNo = 0
  sees 7 -> countYes = 3, countNo = 0
  
```

Good! We got the right answer. But did we just get lucky?

We should pause for a moment to think what this algorithm is doing.

1. We start off with [3] and we expand the subarray until 3 is no longer the majority element. We fail at [3, 1]. At the moment we fail, the subarray can have no majority element.
2. Then we go to [7] and expand until [7, 1]. Again, we terminate and nothing could be the majority element in that subarray.
3. We move to [1] and expand to [1, 7]. We terminate. Nothing there could be the majority element.
4. We go to [7] and expand to [7, 3]. We terminate. Nothing there could be the majority element.
5. We go to [7] and expand until the end of the array: [7, 7, 7]. We have found the majority element (and now we must validate that).

Each time we terminate the `validate` step, the subarray has no majority element. This means that there are at least as many non-7s as there are 7s. Although we're essentially removing this subarray from the original array, the majority element will still be found in the rest of the array—and will still have majority status. Therefore, at some point, we will discover the majority element.

Our algorithm can now be run in two passes: one to find the possible majority element and another to validate it. Rather than using two variables to count (`countYes` and `countNo`), we'll just use a single count variable that increments and decrements.

```

1 int findMajorityElement(int[] array) {
2     int candidate = getCandidate(array);
3     return validate(array, candidate) ? candidate : -1;
4 }
5
6 int getCandidate(int[] array) {
7     int majority = 0;
  
```

```

8     int count = 0;
9     for (int n : array) {
10         if (count == 0) { // No majority element in previous set.
11             majority = n;
12         }
13         if (n == majority) {
14             count++;
15         } else {
16             count--;
17         }
18     }
19     return majority;
20 }
21
22 boolean validate(int[] array, int majority) {
23     int count = 0;
24     for (int n : array) {
25         if (n == majority) {
26             count++;
27         }
28     }
29
30     return count > array.length / 2;
31 }

```

This algorithm runs in $O(N)$ time and $O(1)$ space.

17.11 Word Distance: You have a large text file containing words. Given any two words, find the shortest distance (in terms of number of words) between them in the file. If the operation will be repeated many times for the same file (but different pairs of words), can you optimize your solution?

pg 187

SOLUTION

We will assume for this question that it doesn't matter whether word1 or word2 appears first. This is a question you should ask your interviewer.

To solve this problem, we can traverse the file just once. We remember throughout our traversal where we've last seen word1 and word2, storing the locations in location1 and location2. If the current locations are better than our best known location, we update the best locations.

The code below implements this algorithm.

```

1  LocationPair findClosest(String[] words, String word1, String word2) {
2      LocationPair best = new LocationPair(-1, -1);
3      LocationPair current = new LocationPair(-1, -1);
4      for (int i = 0; i < words.length; i++) {
5          String word = words[i];
6          if (word.equals(word1)) {
7              current.location1 = i;
8              best.updateWithMin(current);
9          } else if (word.equals(word2)) {
10             current.location2 = i;
11             best.updateWithMin(current); // If shorter, update values
12         }
13     }

```



```
14     return best;
15 }
16
17 public class LocationPair {
18     public int location1, location2;
19     public LocationPair(int first, int second) {
20         setLocations(first, second);
21     }
22
23     public void setLocations(int first, int second) {
24         this.location1 = first;
25         this.location2 = second;
26     }
27
28     public void setLocations(LocationPair loc) {
29         setLocations(loc.location1, loc.location2);
30     }
31
32     public int distance() {
33         return Math.abs(location1 - location2);
34     }
35
36     public boolean isValid() {
37         return location1 >= 0 && location2 >= 0;
38     }
39
40     public void updateWithMin(LocationPair loc) {
41         if (!isValid() || loc.distance() < distance()) {
42             setLocations(loc);
43         }
44     }
45 }
```

If we need to repeat the operation for other pairs of words, we can create a hash table that maps from each word to the locations where it occurs. We'll only need to read through the list of words once. After that point, we can do a very similar algorithm but just iterate through the locations directly.

Consider the following lists of locations.

listA: {1, 2, 9, 15, 25}

listB: {4, 10, 19}

Picture pointers pA and pB that point to the beginning of each list. Our goal is to make pA and pB point to values as close together as possible.

The first potential pair is (1, 4).

What is the next pair we can find? If we moved pB, then the distance would definitely get larger. If we moved pA, though, we might get a better pair. Let's do that.

The second potential pair is (2, 4). This is better than the previous pair, so let's record this as the best pair.

We move pA again and get (9, 4). This is worse than we had before.

Now, since the value at pA is bigger than the one at pB, we move pB. We get (9, 10).

Next we get (15, 10), then (15, 19), then (25, 19).

We can implement this algorithm as shown below.

```
1 LocationPair findClosest(String word1, String word2,
```

```

2         HashMapList<String, Integer> locations) {
3     ArrayList<Integer> locations1 = locations.get(word1);
4     ArrayList<Integer> locations2 = locations.get(word2);
5     return findMinDistancePair(locations1, locations2);
6 }
7
8 LocationPair findMinDistancePair(ArrayList<Integer> array1,
9                                 ArrayList<Integer> array2) {
10     if (array1 == null || array2 == null || array1.size() == 0 ||
11         array2.size() == 0) {
12         return null;
13     }
14
15     int index1 = 0;
16     int index2 = 0;
17     LocationPair best = new LocationPair(array1.get(0), array2.get(0));
18     LocationPair current = new LocationPair(array1.get(0), array2.get(0));
19
20     while (index1 < array1.size() && index2 < array2.size()) {
21         current.setLocations(array1.get(index1), array2.get(index2));
22         best.updateWithMin(current); // If shorter, update values
23         if (current.location1 < current.location2) {
24             index1++;
25         } else {
26             index2++;
27         }
28     }
29
30     return best;
31 }
32
33 /* Precomputation. */
34 HashMapList<String, Integer> getWordLocations(String[] words) {
35     HashMapList<String, Integer> locations = new HashMapList<String, Integer>();
36     for (int i = 0; i < words.length; i++) {
37         locations.put(words[i], i);
38     }
39     return locations;
40 }
41
42 /* HashMapList<String, Integer> is a HashMap that maps from Strings to
43    * ArrayList<Integer>. See appendix for implementation. */

```

The precomputation step of this algorithm will take $O(N)$ time, where N is the number of words in the string.

Finding the closest pair of locations will take $O(A + B)$ time, where A is the number of occurrences of the first word and B is the number of occurrences of the second word.

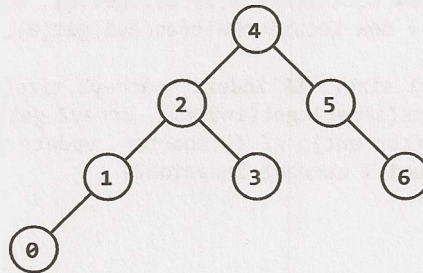
17.12 BiNode: Consider a simple data structure called **BiNode**, which has pointers to two other nodes. The data structure **BiNode** could be used to represent both a binary tree (where **node1** is the left node and **node2** is the right node) or a doubly linked list (where **node1** is the previous node and **node2** is the next node). Implement a method to convert a binary search tree (implemented with **BiNode**) into a doubly linked list. The values should be kept in order and the operation should be performed in place (that is, on the original data structure).

pg 188

SOLUTION

This seemingly complex problem can be implemented quite elegantly using recursion. You will need to understand recursion very well to solve it.

Picture a simple binary search tree:



The `convert` method should transform it into the below doubly linked list:

0 <-> 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6

Let's approach this recursively, starting with the root (node 4).

We know that the left and right halves of the tree form their own "sub-parts" of the linked list (that is, they appear consecutively in the linked list). So, if we recursively converted the left and right subtrees to a doubly linked list, could we build the final linked list from those parts?

Yes! We would simply merge the different parts.

The pseudocode looks something like:

```

1 BiNode convert(BiNode node) {
2     BiNode left = convert(node.left);
3     BiNode right = convert(node.right);
4     mergeLists(left, node, right);
5     return left; // front of left
6 }
  
```

To actually implement the nitty-gritty details of this, we'll need to get the head and tail of each linked list. We can do this several different ways.

Solution #1: Additional Data Structure

The first, and easier, approach is to create a new data structure called **NodePair** which holds just the head and tail of a linked list. The `convert` method can then return something of type **NodePair**.

The code below implements this approach.

```

1 private class NodePair {
  
```

```

2     BiNode head, tail;
3
4     public NodePair(BiNode head, BiNode tail) {
5         this.head = head;
6         this.tail = tail;
7     }
8 }
9
10 public NodePair convert(BiNode root) {
11     if (root == null) return null;
12
13     NodePair part1 = convert(root.node1);
14     NodePair part2 = convert(root.node2);
15
16     if (part1 != null) {
17         concat(part1.tail, root);
18     }
19
20     if (part2 != null) {
21         concat(root, part2.head);
22     }
23
24     return new NodePair(part1 == null ? root : part1.head,
25                         part2 == null ? root : part2.tail);
26 }
27
28 public static void concat(BiNode x, BiNode y) {
29     x.node2 = y;
30     y.node1 = x;
31 }

```

The above code still converts the `BiNode` data structure in place. We're just using `NodePair` as a way to return additional data. We could have alternatively used a two-element `BiNode` array to fulfill the same purposes, but it looks a bit messier (and we like clean code, especially in an interview).

It'd be nice, though, if we could do this without these extra data structures—and we can.

Solution #2: Retrieving the Tail

Instead of returning the head and tail of the linked list with `NodePair`, we can return just the head, and then we can use the head to find the tail of the linked list.

```

1 BiNode convert(BiNode root) {
2     if (root == null) return null;
3
4     BiNode part1 = convert(root.node1);
5     BiNode part2 = convert(root.node2);
6
7     if (part1 != null) {
8         concat(getTail(part1), root);
9     }
10
11    if (part2 != null) {
12        concat(root, part2);
13    }
14
15    return part1 == null ? root : part1;

```

```
16 }
17
18 public static BiNode getTail(BiNode node) {
19     if (node == null) return null;
20     while (node.node2 != null) {
21         node = node.node2;
22     }
23     return node;
24 }
```

Other than a call to `getTail`, this code is almost identical to the first solution. It is not, however, very efficient. A leaf node at depth d will be “touched” by the `getTail` method d times (one for each node above it), leading to an $O(N^2)$ overall runtime, where N is the number of nodes in the tree.

Solution #3: Building a Circular Linked List

We can build our third and final approach off of the second one.

This approach requires returning the head and tail of the linked list with `BiNode`. We can do this by returning each list as the head of a *circular* linked list. To get the tail, then, we simply call `head.node1`.

```
1  BiNode convertToCircular(BiNode root) {
2      if (root == null) return null;
3
4      BiNode part1 = convertToCircular(root.node1);
5      BiNode part3 = convertToCircular(root.node2);
6
7      if (part1 == null && part3 == null) {
8          root.node1 = root;
9          root.node2 = root;
10         return root;
11     }
12     BiNode tail3 = (part3 == null) ? null : part3.node1;
13
14     /* join left to root */
15     if (part1 == null) {
16         concat(part3.node1, root);
17     } else {
18         concat(part1.node1, root);
19     }
20
21     /* join right to root */
22     if (part3 == null) {
23         concat(root, part1);
24     } else {
25         concat(root, part3);
26     }
27
28     /* join right to left */
29     if (part1 != null && part3 != null) {
30         concat(tail3, part1);
31     }
32
33     return part1 == null ? root : part1;
34 }
35
36 /* Convert list to a circular linked list, then break the circular connection. */
```



```

37 BiNode convert(BiNode root) {
38     BiNode head = convertToCircular(root);
39     head.node1.node2 = null;
40     head.node1 = null;
41     return head;
42 }

```

Observe that we have moved the main parts of the code into `convertToCircular`. The `convert` method calls this method to get the head of the circular linked list, and then breaks the circular connection.

The approach takes $O(N)$ time, since each node is only touched an average of once (or, more accurately, $O(1)$ times).

17.13 Re-Space: Oh, no! You have accidentally removed all spaces, punctuation, and capitalization in a lengthy document. A sentence like “I reset the computer. It still didn’t boot!” became “iresetthecomputeritstill didntboot”. You’ll deal with the punctuation and capitalization later; right now you need to re-insert the spaces. Most of the words are in a dictionary but a few are not. Given a dictionary (a list of strings) and the document (a string), design an algorithm to unconcatenate the document in a way that minimizes the number of unrecognized characters.

EXAMPLE

Input: jesslookedjustliketimherbrother

Output: jess looked just like tim her brother (7 unrecognized characters)

pg 188

SOLUTION

Some interviewers like to cut to the chase and give you the specific problems. Others, though, like to give you a lot of unnecessary context, like this problem has. It’s useful in such cases to boil down the problem to what it’s really all about.

In this case, the problem is really about finding a way to break up a string into separate words such that as few characters as possible are “left out” of the parsing.

Note that we do not attempt to “understand” the string. We could just as well parse “thisisawesome” to be “this is a we some” as we could “this is awesome.”

Brute Force

The key to this problem is finding a way to define the solution (that is, parsed string) in terms of its subproblems. One way to do this is recursing through the string.

The very first choice we make is where to insert the first space. After the first character? Second character? Third character?

Let’s imagine this in terms of a string like `thisismikesfavoritefood`. What is the first space we insert?

- If we insert a space after `t`, this gives us one invalid character.
- After `th` is two invalid characters.
- After `thi` is three invalid characters.
- At `this` we have a complete word. This is zero invalid characters.
- At `thisi` is five invalid characters.
- ... and so on.

After we choose the first space, we can recursively pick the second space, then the third space, and so on, until we are done with the string.

We take the best (fewest invalid characters) out of all these choices and return.

What should the function return? We need both the number of invalid characters in the recursive path as well as the actual parsing. Therefore, we just return both by using a custom-built `ParseResult` class.

```

1 String bestSplit(HashSet<String> dictionary, String sentence) {
2     ParseResult r = split(dictionary, sentence, 0);
3     return r == null ? null : r.parsed;
4 }
5
6 ParseResult split(HashSet<String> dictionary, String sentence, int start) {
7     if (start >= sentence.length()) {
8         return new ParseResult(0, "");
9     }
10
11     int bestInvalid = Integer.MAX_VALUE;
12     String bestParsing = null;
13     String partial = "";
14     int index = start;
15     while (index < sentence.length()) {
16         char c = sentence.charAt(index);
17         partial += c;
18         int invalid = dictionary.contains(partial) ? 0 : partial.length();
19         if (invalid < bestInvalid) { // Short circuit
20             /* Recurse, putting a space after this character. If this is better than
21              * the current best option, replace the best option. */
22             ParseResult result = split(dictionary, sentence, index + 1);
23             if (invalid + result.invalid < bestInvalid) {
24                 bestInvalid = invalid + result.invalid;
25                 bestParsing = partial + " " + result.parsed;
26                 if (bestInvalid == 0) break; // Short circuit
27             }
28         }
29
30         index++;
31     }
32     return new ParseResult(bestInvalid, bestParsing);
33 }
34
35 public class ParseResult {
36     public int invalid = Integer.MAX_VALUE;
37     public String parsed = "";
38     public ParseResult(int inv, String p) {
39         invalid = inv;
40         parsed = p;
41     }
42 }

```

We've applied two short circuits here.

- Line 22: If the number of current invalid characters exceeds the best known one, then we know this recursive path will not be ideal. There's no point in even taking it.
- Line 30: If we have a path with zero invalid characters, then we know we can't do better than this. We might as well accept this path.

What's the runtime of this? It's difficult to truly describe in practice as it depends on the (English) language.

One way of looking at it is to imagine a bizarre language where essentially all paths in the recursion are taken. In this case, we are making both choices at each character. If there are n characters, this is an $O(2^n)$ runtime.

Optimized

Commonly, when we have exponential runtimes for a recursive algorithm, we optimize them through memoization (that is, caching results). To do so, we need to find the common subproblems.

Where do recursive paths overlap? That is, where are the common subproblems?

Let's again imagine the string `thisismikesfavoritefood`. Again, imagine that everything is a valid word.

In this case, we attempt to insert the first space after `t` as well as after `th` (and many other choices). Think about what the next choice is.

```
split(thisismikesfavoritefood) ->
    t + split(hisismikesfavoritefood)
    OR th + split(isismikesfavoritefood)
    OR ...

split(hisismikesfavoritefood) ->
    h + split(isismikesfavoritefood)
    OR ...

...
```

Adding a space after `t` and `h` leads to the same recursive path as inserting a space after `th`. There's no sense in computing `split(isismikesfavoritefood)` twice when it will lead to the same result.

We should instead cache the result. We do this using a hash table which maps from the current substring to the `ParseResult` object.

We don't actually need to make the current substring a key. The `start` index in the string sufficiently represents the substring. After all, if we were to use the substring, we'd really be using `sentence.substring(start, sentence.length)`. This hash table will map from a `start` index to the best parsing from that index to the end of the string.

And, since the `start` index is the key, we don't need a true hash table at all. We can just use an array of `ParseResult` objects. This will also serve the purpose of mapping from an index to an object.

The code is essentially identical to the earlier function, but now takes in a memo table (a cache). We look up when we first call the function and set it when we return.

```
1 String bestSplit(HashSet<String> dictionary, String sentence) {
2     ParseResult[] memo = new ParseResult[sentence.length()];
3     ParseResult r = split(dictionary, sentence, 0, memo);
4     return r == null ? null : r.parsed;
5 }
6
7 ParseResult split(HashSet<String> dictionary, String sentence, int start,
8     ParseResult[] memo) {
9     if (start >= sentence.length()) {
10        return new ParseResult(0, "");
11    } if (memo[start] != null) {
12        return memo[start];
13    }
```

```

14
15     int bestInvalid = Integer.MAX_VALUE;
16     String bestParsing = null;
17     String partial = "";
18     int index = start;
19     while (index < sentence.length()) {
20         char c = sentence.charAt(index);
21         partial += c;
22         int invalid = dictionary.contains(partial) ? 0 : partial.length();
23         if (invalid < bestInvalid) { // Short circuit
24             /* Recurse, putting a space after this character. If this is better than
25              * the current best option, replace the best option. */
26             ParseResult result = split(dictionary, sentence, index + 1, memo);
27             if (invalid + result.invalid < bestInvalid) {
28                 bestInvalid = invalid + result.invalid;
29                 bestParsing = partial + " " + result.parsed;
30                 if (bestInvalid == 0) break; // Short circuit
31             }
32         }
33
34         index++;
35     }
36     memo[start] = new ParseResult(bestInvalid, bestParsing);
37     return memo[start];
38 }

```

Understanding the runtime of this is even trickier than in the prior solution. Again, let's imagine the truly bizarre case, where essentially everything looks like a valid word.

One way we can approach it is to realize that `split(i)` will only be computed once for each value of `i`. What happens when we call `split(i)`, assuming we've already called `split(i+1)` through `split(n - 1)`?

```

split(i) -> calls:
    split(i + 1)
    split(i + 2)
    split(i + 3)
    split(i + 4)
    ...
    split(n - 1)

```

Each of the recursive calls has already been computed, so they just return immediately. Doing `n - i` calls at $O(1)$ time each takes $O(n - i)$ time. This means that `split(i)` takes $O(i)$ time at most.

We can now apply the same logic to `split(i - 1)`, `split(i - 2)`, and so on. If we make 1 call to compute `split(n - 1)`, 2 calls to compute `split(n - 2)`, 3 calls to compute `split(n - 3)`, ..., `n` calls to compute `split(0)`, how many calls total do we do? This is basically the sum of the numbers from 1 through `n`, which is $O(n^2)$.

Therefore, the runtime of this function is $O(n^2)$.

17.14 Smallest K: Design an algorithm to find the smallest K numbers in an array.

pg 188

SOLUTION

There are a number of ways to approach this problem. We will go through three of them: sorting, max heap, and selection rank.

Some of these algorithms require modifying the array. This is something you should discuss with your interviewer. Note, though, that even if modifying the original array is not acceptable, you can always clone the array and modify the clone instead. This will not impact the overall big O time of any algorithm.

Solution 1: Sorting

We can sort the elements in ascending order and then take the first million numbers from that.

```
1  int[] smallestK(int[] array, int k) {
2      if (k <= 0 || k > array.length) {
3          throw new IllegalArgumentException();
4      }
5
6      /* Sort array. */
7      Arrays.sort(array);
8
9      /* Copy first k elements. */
10     int[] smallest = new int[k];
11     for (int i = 0; i < k; i++) {
12         smallest[i] = array[i];
13     }
14     return smallest;
15 }
```

The time complexity is $O(n \log(n))$.

Solution 2: Max Heap

We can use a max heap to solve this problem. We first create a max heap (largest element at the top) for the first million numbers.

Then, we traverse through the list. On each element, if it's smaller than the root, we insert it into the heap and delete the largest element (which will be the root).

At the end of the traversal, we will have a heap containing the smallest one million numbers. This algorithm is $O(n \log(m))$, where m is the number of values we are looking for.

```
1  int[] smallestK(int[] array, int k) {
2      if (k <= 0 || k > array.length) {
3          throw new IllegalArgumentException();
4      }
5
6      PriorityQueue<Integer> heap = getKMaxHeap(array, k);
7      return heapToIntArray(heap);
8  }
9
10 /* Create max heap of smallest k elements. */
11 PriorityQueue<Integer> getKMaxHeap(int[] array, int k) {
12     PriorityQueue<Integer> heap =
```



```

13     new PriorityQueue<Integer>(k, new MaxHeapComparator());
14     for (int a : array) {
15         if (heap.size() < k) { // If space remaining
16             heap.add(a);
17         } else if (a < heap.peek()) { // If full and top is small
18             heap.poll(); // remove highest
19             heap.add(a); // insert new element
20         }
21     }
22     return heap;
23 }
24
25 /* Convert heap to int array. */
26 int[] heapToIntArray(PriorityQueue<Integer> heap) {
27     int[] array = new int[heap.size()];
28     while (!heap.isEmpty()) {
29         array[heap.size() - 1] = heap.poll();
30     }
31     return array;
32 }
33
34 class MaxHeapComparator implements Comparator<Integer> {
35     public int compare(Integer x, Integer y) {
36         return y - x;
37     }
38 }

```

Java's uses the `PriorityQueue` class to offer heap-like functionality. By default, it operates as a min heap, with the smallest element on the top. To switch it to the biggest element on the top, we can pass in a different comparator.

Approach 3: Selection Rank Algorithm (if elements are unique)

Selection Rank is a well-known algorithm in computer science to find the i th smallest (or largest) element in an array in linear time.

If the elements are unique, you can find the i th smallest element in expected $O(n)$ time. The basic algorithm operates like this:

1. Pick a random element in the array and use it as a "pivot." Partition elements around the pivot, keeping track of the number of elements on the left side of the partition.
2. If there are exactly i elements on the left, then you just return the biggest element on the left.
3. If the left side is bigger than i , repeat the algorithm on just the left part of the array.
4. If the left side is smaller than i , repeat the algorithm on the right, but look for the element with rank $i - \text{leftSize}$.

Once you have found the i th smallest element, you know that all elements smaller than this will be to the left of this (since you've partitioned the array accordingly). You can now just return the first i elements.

The code below implements this algorithm.

```

1  int[] smallestK(int[] array, int k) {
2      if (k <= 0 || k > array.length) {
3          throw new IllegalArgumentException();
4      }
5  }

```

```

6   int threshold = rank(array, k - 1);
7   int[] smallest = new int[k];
8   int count = 0;
9   for (int a : array) {
10      if (a <= threshold) {
11         smallest[count] = a;
12         count++;
13      }
14  }
15  return smallest;
16 }
17
18 /* Get element with rank. */
19 int rank(int[] array, int rank) {
20     return rank(array, 0, array.length - 1, rank);
21 }
22
23 /* Get element with rank between left and right indices. */
24 int rank(int[] array, int left, int right, int rank) {
25     int pivot = array[randomIntInRange(left, right)];
26     int leftEnd = partition(array, left, right, pivot);
27     int leftSize = leftEnd - left + 1;
28     if (rank == leftSize - 1) {
29         return max(array, left, leftEnd);
30     } else if (rank < leftSize) {
31         return rank(array, left, leftEnd, rank);
32     } else {
33         return rank(array, leftEnd + 1, right, rank - leftSize);
34     }
35 }
36
37 /* Partition array around pivot such that all elements <= pivot come before all
38  * elements > pivot. */
39 int partition(int[] array, int left, int right, int pivot) {
40     while (left <= right) {
41         if (array[left] > pivot) {
42             /* Left is bigger than pivot. Swap it to the right side, where we know it
43              * should be. */
44             swap(array, left, right);
45             right--;
46         } else if (array[right] <= pivot) {
47             /* Right is smaller than the pivot. Swap it to the left side, where we know
48              * it should be. */
49             swap(array, left, right);
50             left++;
51         } else {
52             /* Left and right are in correct places. Expand both sides. */
53             left++;
54             right--;
55         }
56     }
57     return left - 1;
58 }
59
60 /* Get random integer within range, inclusive. */
61 int randomIntInRange(int min, int max) {

```

```

62 Random rand = new Random();
63 return rand.nextInt(max + 1 - min) + min;
64 }
65
66 /* Swap values at index i and j. */
67 void swap(int[] array, int i, int j) {
68     int t = array[i];
69     array[i] = array[j];
70     array[j] = t;
71 }
72
73 /* Get largest element in array between left and right indices. */
74 int max(int[] array, int left, int right) {
75     int max = Integer.MIN_VALUE;
76     for (int i = left; i <= right; i++) {
77         max = Math.max(array[i], max);
78     }
79     return max;
80 }

```

If the elements are not unique, we can tweak this algorithm slightly to accommodate this.

Approach 4: Selection Rank Algorithm (if elements are not unique)

The major change that needs to be made is to the partition function. When we partition the array around a pivot element, we now partition it into three chunks: less than pivot, equal to pivot, and greater than pivot.

This requires minor tweaks to rank as well. We now compare the size of left and middle partitions to rank.

```

1 class PartitionResult {
2     int leftSize, middleSize;
3     public PartitionResult(int left, int middle) {
4         this.leftSize = left;
5         this.middleSize = middle;
6     }
7 }
8
9 int[] smallestK(int[] array, int k) {
10     if (k <= 0 || k > array.length) {
11         throw new IllegalArgumentException();
12     }
13
14     /* Get item with rank k - 1. */
15     int threshold = rank(array, k - 1);
16
17     /* Copy elements smaller than the threshold element. */
18     int[] smallest = new int[k];
19     int count = 0;
20     for (int a : array) {
21         if (a < threshold) {
22             smallest[count] = a;
23             count++;
24         }
25     }
26
27     /* If there's still room left, this must be for elements equal to the threshold

```

```

28     * element. Copy those in. */
29     while (count < k) {
30         smallest[count] = threshold;
31         count++;
32     }
33
34     return smallest;
35 }
36
37 /* Find value with rank k in array. */
38 int rank(int[] array, int k) {
39     if (k >= array.length) {
40         throw new IllegalArgumentException();
41     }
42     return rank(array, k, 0, array.length - 1);
43 }
44
45 /* Find value with rank k in sub array between start and end. */
46 int rank(int[] array, int k, int start, int end) {
47     /* Partition array around an arbitrary pivot. */
48     int pivot = array[randomIntInRange(start, end)];
49     PartitionResult partition = partition(array, start, end, pivot);
50     int leftSize = partition.leftSize;
51     int middleSize = partition.middleSize;
52
53     /* Search portion of array. */
54     if (k < leftSize) { // Rank k is on left half
55         return rank(array, k, start, start + leftSize - 1);
56     } else if (k < leftSize + middleSize) { // Rank k is in middle
57         return pivot; // middle is all pivot values
58     } else { // Rank k is on right
59         return rank(array, k - leftSize - middleSize, start + leftSize + middleSize,
60                     end);
61     }
62 }
63
64 /* Partition result into < pivot, equal to pivot -> bigger than pivot. */
65 PartitionResult partition(int[] array, int start, int end, int pivot) {
66     int left = start; /* Stays at (right) edge of left side. */
67     int right = end; /* Stays at (left) edge of right side. */
68     int middle = start; /* Stays at (right) edge of middle. */
69     while (middle <= right) {
70         if (array[middle] < pivot) {
71             /* Middle is smaller than the pivot. Left is either smaller or equal to
72              * the pivot. Either way, swap them. Then middle and left should move by
73              * one. */
74             swap(array, middle, left);
75             middle++;
76             left++;
77         } else if (array[middle] > pivot) {
78             /* Middle is bigger than the pivot. Right could have any value. Swap them,
79              * then we know that the new right is bigger than the pivot. Move right by
80              * one. */
81             swap(array, middle, right);
82             right--;
83         } else if (array[middle] == pivot) {

```

```

84         /* Middle is equal to the pivot. Move by one. */
85         middle++;
86     }
87 }
88
89 /* Return sizes of left and middle. */
90 return new PartitionResult(left - start, right - left + 1);
91 }

```

Notice the change made to `smallestK` too. We can't simply copy all elements less than or equal to `threshold` into the array. Since we have duplicates, there could be many more than `k` elements that are less than or equal to `threshold`. (We also can't just say "okay, only copy `k` elements over." We could inadvertently fill up the array early on with "equal" elements, and not leave enough space for the smaller ones.)

The solution for this is fairly simple: only copy over the smaller elements first, then fill up the array with equal elements at the end.

17.15 Longest Word: Given a list of words, write a program to find the longest word made of other words in the list.

pg 188

SOLUTION

This problem seems complex, so let's simplify it. What if we just wanted to know the longest word made of *two* other words in the list?

We could solve this by iterating through the list, from the longest word to the shortest word. For each word, we would split it into all possible pairs and check if both the left and right side are contained in the list.

The pseudocode for this would look like the following:

```

1  String getLongestWord(String[] list) {
2      String[] array = list.SortByLength();
3      /* Create map for easy lookup */
4      HashMap<String, Boolean> map = new HashMap<String, Boolean>;
5
6      for (String str : array) {
7          map.put(str, true);
8      }
9
10     for (String s : array) {
11         // Divide into every possible pair
12         for (int i = 1; i < s.length(); i++) {
13             String left = s.substring(0, i);
14             String right = s.substring(i);
15             // Check if both sides are in the array
16             if (map[left] == true && map[right] == true) {
17                 return s;
18             }
19         }
20     }
21     return str;
22 }

```

This works great for when we just want to know composites of two words. But what if a word could be formed by any number of other words?

In this case, we could apply a very similar approach, with one modification: rather than simply looking up if the right side is in the array, we would recursively see if we can build the right side from the other elements in the array.

The code below implements this algorithm:

```

1 String printLongestWord(String arr[]) {
2     HashMap<String, Boolean> map = new HashMap<String, Boolean>();
3     for (String str : arr) {
4         map.put(str, true);
5     }
6     Arrays.sort(arr, new LengthComparator()); // Sort by length
7     for (String s : arr) {
8         if (canBuildWord(s, true, map)) {
9             System.out.println(s);
10            return s;
11        }
12    }
13    return "";
14 }

15
16 boolean canBuildWord(String str, boolean isOriginalWord,
17                      HashMap<String, Boolean> map) {
18     if (map.containsKey(str) && !isOriginalWord) {
19         return map.get(str);
20     }
21     for (int i = 1; i < str.length(); i++) {
22         String left = str.substring(0, i);
23         String right = str.substring(i);
24         if (map.containsKey(left) && map.get(left) == true &&
25             canBuildWord(right, false, map)) {
26             return true;
27         }
28     }
29     map.put(str, false);
30     return false;
31 }

```

Note that in this solution we have performed a small optimization. We use a dynamic programming/memoization approach to cache the results between calls. This way, if we repeatedly need to check if there's any way to build "testingtester," we'll only have to compute it once.

A boolean flag `isOriginalWord` is used to complete the above optimization. The method `canBuildWord` is called for the original word and for each substring, and its first step is to check the cache for a previously calculated result. However, for the original words, we have a problem: `map` is initialized to `true` for them, but we don't want to return `true` (since a word cannot be composed solely of itself). Therefore, for the original word, we simply bypass this check using the `isOriginalWord` flag.

17.16 The Masseuse: A popular masseuse receives a sequence of back-to-back appointment requests and is debating which ones to accept. She needs a 15-minute break between appointments and therefore she cannot accept any adjacent requests. Given a sequence of back-to-back appointment requests (all multiples of 15 minutes, none overlap, and none can be moved), find the optimal (highest total booked minutes) set the masseuse can honor. Return the number of minutes.

EXAMPLE

Input: {30, 15, 60, 75, 45, 15, 15, 45}

Output: 180 minutes ({30, 60, 45, 45}).

pg 188

SOLUTION

Let's start with an example. We'll draw it visually to get a better feel for the problem. Each number indicates the number of minutes in the appointment.

$r_0 = 75$	$r_1 = 105$	$r_2 = 120$	$r_3 = 75$	$r_4 = 90$	$r_5 = 135$
------------	-------------	-------------	------------	------------	-------------

Alternatively, we could have also divided all the values (including the break) by 15 minutes, to give us the array {5, 7, 8, 5, 6, 9}. This would be equivalent, but now we would want a 1-minute break.

The best set of appointments for this problem has 330 minutes total, formed with $\{r_0 = 75, r_2 = 120, r_5 = 135\}$. Note that we've intentionally chosen an example in which the best sequence of appointments was not formed through a strictly alternating sequence.

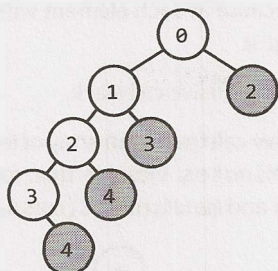
We should also recognize that choosing the longest appointment first (the "greedy" strategy) would not necessarily be optimal. For example, a sequence like {45, 60, 45, 15} would not have 60 in the optimal set.

Solution #1: Recursion

The first thing that may come to mind is a recursive solution. We have essentially a sequence of choices as we walk down the list of appointments: Do we use this appointment or do we not? If we use appointment i , we must skip appointment $i + 1$ as we can't take back-to-back appointments. Appointment $i + 2$ is a possibility (but not necessarily the best choice).

```

1  int maxMinutes(int[] messages) {
2      return maxMinutes(messages, 0);
3  }
4
5  int maxMinutes(int[] messages, int index) {
6      if (index >= messages.length) { // Out of bounds
7          return 0;
8      }
9
10     /* Best with this reservation. */
11     int bestWith = messages[index] + maxMinutes(messages, index + 2);
12
13     /* Best without this reservation. */
14     int bestWithout = maxMinutes(messages, index + 1);
15
16     /* Return best of this subarray, starting from index. */
17     return Math.max(bestWith, bestWithout);
18 }
```

If we drew a bigger tree, we'd see a similar pattern. The tree looks very linear, with one branch down to the left. This gives us an $O(n)$ runtime and $O(n)$ space. The space usage comes from the recursive call stack as well as from the memo table.

Solution #3: Iterative

Can we do better? We certainly can't beat the time complexity since we have to look at each appointment. However, we might be able to beat the space complexity. This would mean not solving the problem recursively.

Let's look at our first example again.

$r_0 = 30$	$r_1 = 15$	$r_2 = 60$	$r_3 = 75$	$r_4 = 45$	$r_5 = 15$	$r_6 = 15$	$r_7 = 45$
------------	------------	------------	------------	------------	------------	------------	------------

As we noted in the problem statement, we cannot take adjacent appointments.

There's another observation, though, that we can make: We should never *skip* three consecutive appointments. That is, we might skip r_1 and r_2 if we wanted to take r_0 and r_3 . But we would never skip r_1 , r_2 , and r_3 . This would be suboptimal since we could always improve our set by grabbing that middle element.

This means that if we take r_0 , we know we'll definitely skip r_1 and definitely take either r_2 or r_3 . This substantially limits the options we need to evaluate and opens the door to an iterative solution.

Let's think about our recursive + memoization solution and try to reverse the logic; that is, let's try to approach it iteratively.

A useful way to do this is to approach it from the back and move toward the start of the array. At each point, we find the solution for the subarray.

- **best(7):** What's the best option for $\{r_7 = 45\}$? We can get 45 min. if we take r_7 , so **best(7) = 45**
- **best(6):** What's the best option for $\{r_6 = 15, \dots\}$? Still 45 min., so **best(6) = 45**.
- **best(5):** What's the best option for $\{r_5 = 15, \dots\}$? We can either:
 - » take $r_5 = 15$ and merge it with **best(7) = 45**, or:
 - » take **best(6) = 45**.

The first gives us 60 minutes, **best(5) = 60**.

- **best(4):** What's the best option for $\{r_4 = 45, \dots\}$? We can either:
 - » take $r_4 = 45$ and merge it with **best(6) = 45**, or:
 - » take **best(5) = 60**.

The first gives us 90 minutes, **best(4) = 90**.

- **best(3):** What's the best option for $\{r_3 = 75, \dots\}$? We can either:
 - » take $r_3 = 75$ and merge it with **best(5) = 60**, or:

» take $\text{best}(4) = 90$.

The first gives us 135 minutes, $\text{best}(3) = 135$.

- $\text{best}(2)$: What's the best option for $\{r_2 = 60, \dots\}$? We can either:

» take $r_2 = 60$ and merge it with $\text{best}(4) = 90$, or:

» take $\text{best}(3) = 135$.

The first gives us 150 minutes, $\text{best}(2) = 150$.

- $\text{best}(1)$: What's the best option for $\{r_1 = 15, \dots\}$? We can either:

» take $r_1 = 15$ and merge it with $\text{best}(3) = 135$, or:

» take $\text{best}(2) = 150$.

Either way, $\text{best}(1) = 150$.

- $\text{best}(0)$: What's the best option for $\{r_0 = 30, \dots\}$? We can either:

» take $r_0 = 30$ and merge it with $\text{best}(2) = 150$, or:

» take $\text{best}(1) = 150$.

The first gives us 180 minutes, $\text{best}(0) = 180$.

Therefore, we return 180 minutes.

The code below implements this algorithm.

```
1 int maxMinutes(int[] messages) {
2     /* Allocating two extra slots in the array so we don't have to do bounds
3      * checking on lines 7 and 8. */
4     int[] memo = new int[messages.length + 2];
5     memo[messages.length] = 0;
6     memo[messages.length + 1] = 0;
7     for (int i = messages.length - 1; i >= 0; i--) {
8         int bestWith = messages[i] + memo[i + 2];
9         int bestWithout = memo[i + 1];
10        memo[i] = Math.max(bestWith, bestWithout);
11    }
12    return memo[0];
13 }
```

The runtime of this solution is $O(n)$ and the space complexity is also $O(n)$.

It's nice in some ways that it's iterative, but we haven't actually "won" anything here. The recursive solution had the same time and space complexity.

Solution #4: Iterative with Optimal Time and Space

In reviewing the last solution, we can recognize that we only use the values in the memo table for a short amount of time. Once we are several elements past an index, we never use that element's index again.

In fact, at any given index i , we only need to know the best value from $i + 1$ and $i + 2$. Therefore, we can get rid of the memo table and just use two integers.

```
1 int maxMinutes(int[] messages) {
2     int oneAway = 0;
3     int twoAway = 0;
4     for (int i = messages.length - 1; i >= 0; i--) {
5         int bestWith = messages[i] + twoAway;
6         int bestWithout = oneAway;
```



```

7         int current = Math.max(bestWith, bestWithout);
8         twoAway = oneAway;
9         oneAway = current;
10    }
11    return oneAway;
12 }

```

This gives us the most optimal time and space possible: $O(n)$ time and $O(1)$ space.

Why did we look backward? It's a common technique in many problems to walk backward through an array.

However, we can walk forward if we want. This is easier for some people to think about, and harder for others. In this case, rather than asking "What's the best set that starts with $a[i]$?", we would ask "What's the best set that ends with $a[i]$?"

17.17 Multi Search: Given a string b and an array of smaller strings T , design a method to search b for each small string in T .

pg 189

SOLUTION

Let's start with an example:

```

T = {"is", "ppi", "hi", "sis", "i", "ssippi"}
b = "mississippi"

```

Note that in our example, we made sure to have some strings (like "is") that appear multiple times in b .

Solution #1

The naive solution is reasonably straightforward. Just search through the bigger string for each instance of the smaller string.

```

1  HashMapList<String, Integer> searchAll(String big, String[] smalls) {
2      HashMapList<String, Integer> lookup =
3          new HashMapList<String, Integer>();
4      for (String small : smalls) {
5          ArrayList<Integer> locations = search(big, small);
6          lookup.put(small, locations);
7      }
8      return lookup;
9  }
10
11  /* Find all locations of the smaller string within the bigger string. */
12  ArrayList<Integer> search(String big, String small) {
13      ArrayList<Integer> locations = new ArrayList<Integer>();
14      for (int i = 0; i < big.length() - small.length() + 1; i++) {
15          if (isSubstringAtLocation(big, small, i)) {
16              locations.add(i);
17          }
18      }
19      return locations;
20  }
21
22  /* Check if small appears at index offset within big. */
23  boolean isSubstringAtLocation(String big, String small, int offset) {
24      for (int i = 0; i < small.length(); i++) {
25          if (big.charAt(offset + i) != small.charAt(i)) {

```

```

26         return false;
27     }
28 }
29 return true;
30 }
31
32 /* HashMapList<String, Integer> is a HashMap that maps from Strings to
33    * ArrayList<Integer>. See appendix for implementation. */

```

We could have also used a `substring` and `equals` function, instead of writing `isAtLocation`. This is slightly faster (though not in terms of big O) because it doesn't require creating a bunch of substrings.

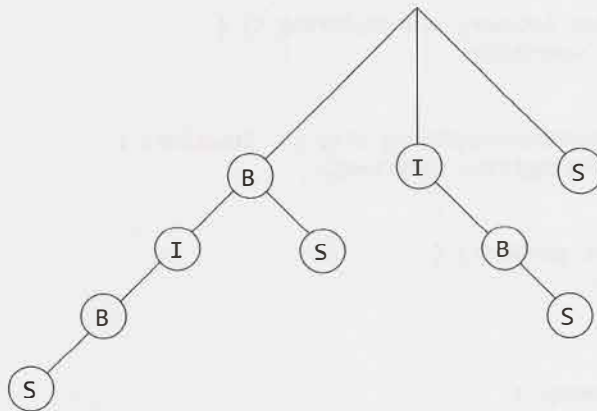
This will take $O(kbt)$ time, where k is the length of the longest string in T , b is the length of the bigger string, and t is the number of smaller strings within T .

Solution #2

To optimize this, we should think about how we can tackle all the elements in T at once, or somehow re-use work.

One way is to create a trie-like data structure using each suffix in the bigger string. For the string `bibs`, the suffix list would be: `bibs`, `ibs`, `bs`, `s`.

The tree for this is below.



Then, all you need to do is search in the suffix tree for each string in T . Note that if "B" were a word, you would come up with two locations.

```

1  HashMapList<String, Integer> searchAll(String big, String[] smalls) {
2      HashMapList<String, Integer> lookup = new HashMapList<String, Integer>();
3      Trie tree = createTrieFromString(big);
4      for (String s : smalls) {
5          /* Get terminating location of each occurrence.*/
6          ArrayList<Integer> locations = tree.search(s);
7
8          /* Adjust to starting location. */
9          subtractValue(locations, s.length());
10
11         /* Insert. */
12         lookup.put(s, locations);
13     }
14     return lookup;

```

```
15 }
16
17 Trie createTrieFromString(String s) {
18     Trie trie = new Trie();
19     for (int i = 0; i < s.length(); i++) {
20         String suffix = s.substring(i);
21         trie.insertString(suffix, i);
22     }
23     return trie;
24 }
25
26 void subtractValue(ArrayList<Integer> locations, int delta) {
27     if (locations == null) return;
28     for (int i = 0; i < locations.size(); i++) {
29         locations.set(i, locations.get(i) - delta);
30     }
31 }
32
33 public class Trie {
34     private TrieNode root = new TrieNode();
35
36     public Trie(String s) { insertString(s, 0); }
37     public Trie() {}
38
39     public ArrayList<Integer> search(String s) {
40         return root.search(s);
41     }
42
43     public void insertString(String str, int location) {
44         root.insertString(str, location);
45     }
46
47     public TrieNode getRoot() {
48         return root;
49     }
50 }
51
52 public class TrieNode {
53     private HashMap<Character, TrieNode> children;
54     private ArrayList<Integer> indexes;
55     private char value;
56
57     public TrieNode() {
58         children = new HashMap<Character, TrieNode>();
59         indexes = new ArrayList<Integer>();
60     }
61
62     public void insertString(String s, int index) {
63         indexes.add(index);
64         if (s != null && s.length() > 0) {
65             value = s.charAt(0);
66             TrieNode child = null;
67             if (children.containsKey(value)) {
68                 child = children.get(value);
69             } else {
70                 child = new TrieNode();
```

```

71         children.put(value, child);
72     }
73     String remainder = s.substring(1);
74     child.insertString(remainder, index + 1);
75 } else {
76     children.put('\0', null); // Terminating character
77 }
78 }
79
80 public ArrayList<Integer> search(String s) {
81     if (s == null || s.length() == 0) {
82         return indexes;
83     } else {
84         char first = s.charAt(0);
85         if (children.containsKey(first)) {
86             String remainder = s.substring(1);
87             return children.get(first).search(remainder);
88         }
89     }
90     return null;
91 }
92
93 public boolean terminates() {
94     return children.containsKey('\0');
95 }
96
97 public TrieNode getChild(char c) {
98     return children.get(c);
99 }
100 }
101
102 /* HashMapList<String, Integer> is a HashMap that maps from Strings to
103 * ArrayList<Integer>. See appendix for implementation. */

```

It takes $O(b^2)$ time to create the tree and $O(kt)$ time to search for the locations.

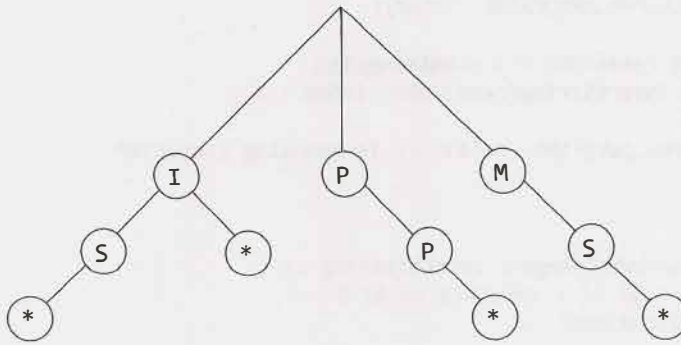
Reminder: k is the length of the longest string in T , b is the length of the bigger string, and t is the number of smaller strings within T .

The total runtime is $O(b^2 + kt)$.

Without some additional knowledge of the expected input, you cannot directly compare $O(bkt)$, which was the runtime of the prior solution, to $O(b^2 + kt)$. If b is very large, then $O(bkt)$ is preferable. But if you have a lot of smaller strings, then $O(b^2 + kt)$ might be better.

Solution #3

Alternatively, we can add all the smaller strings into a trie. For example, the strings {i, is, pp, ms} would look like the trie below. The asterisk (*) hanging from a node indicates that this node completes a word.



Now, when we want to find all words in `mississippi`, we search through this trie starting with each word.

- **m:** We would first look up in the trie starting with `m`, the first letter in `mississippi`. As soon as we go to `mi`, we terminate.
- **i:** Then, we go to `i`, the second character in `mississippi`. We see that `i` is a complete word, so we add it to the list. We also keep going with `i` over to `is`. The string `is` is also a complete word, so we add that to the list. This node has no more children, so we move onto the next character in `mississippi`.
- **s:** We now go to `s`. There is no upper-level node for `s`, so we go onto the next character.
- **s:** Another `s`. Go on to the next character.
- **i:** We see another `i`. We go to the `i` node in the trie. We see that `i` is a complete word, so we add it to the list. We also keep going with `i` over to `is`. The string `is` is also a complete word, so we add that to the list. This node has no more children, so we move onto the next character in `mississippi`.
- **s:** We go to `s`. There is no upper-level node for `s`.
- **s:** Another `s`. Go on to the next character.
- **i:** We go to the `i` node. We see that `i` is a complete word, so we add it to the trie. The next character in `mississippi` is a `p`. There is no node `p`, so we break here.
- **p:** We see a `p`. There is no node `p`.
- **p:** Another `p`.
- **i:** We go to the `i` node. We see that `i` is a complete word, so we add it to the trie. There are no more characters left in `mississippi`, so we are done.

Each time we find a complete “small” word, we add it to a list along with the location in the bigger word (`mississippi`) where we found the small word.

The code below implements this algorithm.

```

1  HashMapList<String, Integer> searchAll(String big, String[] smalls) {
2      HashMapList<String, Integer> lookup = new HashMapList<String, Integer>();
3      int maxLen = big.length();
4      TrieNode root = createTreeFromStrings(smalls, maxLen).getRoot();
5
6      for (int i = 0; i < big.length(); i++) {
7          ArrayList<String> strings = findStringsAtLoc(root, big, i);
8          insertIntoHashMap(strings, lookup, i);
9      }
10
11     return lookup;

```



```

12 }
13
14 /* Insert each string into trie (provided string is not longer than maxLen). */
15 Trie createTreeFromStrings(String[] smalls, int maxLen) {
16     Trie tree = new Trie("");
17     for (String s : smalls) {
18         if (s.length() <= maxLen) {
19             tree.insertString(s, 0);
20         }
21     }
22     return tree;
23 }
24
25 /* Find strings in trie that start at index "start" within big. */
26 ArrayList<String> findStringsAtLoc(TrieNode root, String big, int start) {
27     ArrayList<String> strings = new ArrayList<String>();
28     int index = start;
29     while (index < big.length()) {
30         root = root.getChild(big.charAt(index));
31         if (root == null) break;
32         if (root.terminates()) { // Is complete string, add to list
33             strings.add(big.substring(start, index + 1));
34         }
35         index++;
36     }
37     return strings;
38 }
39
40 /* HashMapList<String, Integer> is a HashMap that maps from Strings to
41    * ArrayList<Integer>. See appendix for implementation. */

```

This algorithm takes $O(kt)$ time to create the trie and $O(bk)$ time to search for all the strings.

■ Reminder: k is the length of the longest string in T , b is the length of the bigger string, and t is the number of smaller strings within T .

The total time to solve the question is $O(kt + bk)$.

Solution #1 was $O(kbt)$. We know that $O(kt + bk)$ will be faster than $O(kbt)$.

Solution #2 was $O(b^2 + kt)$. Since b will always be bigger than k (or if it's not, then we know this really long string k cannot be found in b), we know Solution #3 is also faster than Solution #2.

17.18 Shortest Supersequence: You are given two arrays, one shorter (with all distinct elements) and one longer. Find the shortest subarray in the longer array that contains all the elements in the shorter array. The items can appear in any order.

EXAMPLE

Input:

{1, 5, 9}

{7, 5, 9, 0, 2, 1, 3, 5, 7, 9, 1, 1, 5, 8, 8, 9, 7}

Output: [7, 10] (the underlined portion above)

pg 189

SOLUTIONS

As usual, a brute force approach is a good way to start. Try thinking about it as if you were doing it by hand. How would you do it?

Let's use the example from the problem to walk through this. We'll call the smaller array `smallArray` and the bigger array `bigArray`.

Brute Force

The slow, "easy" way to do this is to iterate through `bigArray` and do repeated small passes through it.

At each index in `bigArray`, scan forward to find the next occurrence of each element in `smallArray`. The largest of these next occurrences will tell us the shortest subarray that starts at that index. (We'll call this concept "closure." That is, the closure is the element that "closes" a complete subarray starting at that index. For example, the closure of index 3—which has value 0—in the example is index 9.)

By finding the closures for each index in the array, we can find the shortest subarray overall.

```

1  Range shortestSupersequence(int[] bigArray, int[] smallArray) {
2      int bestStart = -1;
3      int bestEnd = -1;
4      for (int i = 0; i < bigArray.length; i++) {
5          int end = findClosure(bigArray, smallArray, i);
6          if (end == -1) break;
7          if (bestStart == -1 || end - i < bestEnd - bestStart) {
8              bestStart = i;
9              bestEnd = end;
10         }
11     }
12     return new Range(bestStart, bestEnd);
13 }
14
15 /* Given an index, find the closure (i.e., the element which terminates a complete
16  * subarray containing all elements in smallArray). This will be the max of the
17  * next locations of each element in smallArray. */
18 int findClosure(int[] bigArray, int[] smallArray, int index) {
19     int max = -1;
20     for (int i = 0; i < smallArray.length; i++) {
21         int next = findNextInstance(bigArray, smallArray[i], index);
22         if (next == -1) {
23             return -1;
24         }
25         max = Math.max(next, max);
26     }

```

```

27     return max;
28 }
29
30 /* Find next instance of element starting from index. */
31 int findNextInstance(int[] array, int element, int index) {
32     for (int i = index; i < array.length; i++) {
33         if (array[i] == element) {
34             return i;
35         }
36     }
37     return -1;
38 }
39
40 public class Range {
41     private int start;
42     private int end;
43     public Range(int s, int e) {
44         start = s;
45         end = e;
46     }
47
48     public int length() { return end - start + 1; }
49     public int getStart() { return start; }
50     public int getEnd() { return end; }
51
52     public boolean shorterThan(Range other) {
53         return length() < other.length();
54     }
55 }

```

This algorithm will potentially take $O(SB^2)$ time, where B is the length of `bigString` and S is the length of `smallString`. This is because at each of the B characters, we potentially do $O(SB)$ work: S scans of the rest of the string, which has potentially B characters.

Optimized

Let's think about how we can optimize this. The core reason why it's slow is the repeated searches. Is there a faster way that we can find, given an index, the next occurrence of a particular character?

Let's think about it with an example. Given the array below, is there a way we could quickly find the next 5 from each location?

7, 5, 9, 0, 2, 1, 3, 5, 7, 9, 1, 1, 5, 8, 8, 9, 7

Yes. Because we're going to have to do this repeatedly, we can precompute this information in just a single (backwards) sweep. Iterate through the array backwards, tracking the last (most recent) occurrence of 5.

value	7	5	9	0	2	1	3	5	7	9	1	1	5	8	8	9	7
index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
next 5	1	1	7	7	7	7	7	7	12	12	12	12	12	x	x	x	x

Doing this for each of {1, 5, 9} takes just 3 backwards sweeps.

Some people want to merge this into one backwards sweep that handles all three values. It feels faster—but it's not really. Doing it in one backwards sweep means doing three comparisons at each iteration. N moves through the list with three comparisons at each move is no better than $3N$ moves and one comparison at each move. You might as well keep the code clean by doing it in separate sweeps.

value	7	5	9	0	2	1	3	5	7	9	1	1	5	8	8	9	7
index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
next 1	5	5	5	5	5	5	10	10	10	10	10	11	x	x	x	x	x
next 5	1	1	7	7	7	7	7	7	12	12	12	12	12	x	x	x	x
next 9	2	2	2	9	9	9	9	9	9	9	15	15	15	15	15	15	x

The findNextInstance function can now just use this table to find the next occurrence, rather than doing a search.

But, actually, we can make it a bit simpler. Using the table above, we can quickly compute the closure of each index. It's just the max of the column. If a column has an x in it, then there is no closure, at this indicates that there's no next occurrence of that character.

The difference between the index and the closure is the smallest subarray starting at that index.

value	7	5	9	0	2	1	3	5	7	9	1	1	5	8	8	9	7
index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
next 1	5	5	5	5	5	5	10	10	10	10	10	11	x	x	x	x	x
next 5	1	1	7	7	7	7	7	7	12	12	12	12	12	x	x	x	x
next 9	2	2	2	9	9	9	9	9	9	9	15	15	15	15	15	15	x
closure	5	5	7	9	9	9	10	10	12	12	15	15	x	x	x	x	x
diff.	5	4	5	6	5	4	4	3	4	3	5	4	x	x	x	x	x

Now, all we have to do is to find the minimum distance in this table.

```
1 Range shortestSupersequence(int[] big, int[] small) {
2     int[][] nextElements = getNextElementsMulti(big, small);
3     int[] closures = getClosures(nextElements);
4     return getShortestClosure(closures);
5 }
6
7 /* Create table of next occurrences. */
8 int[][] getNextElementsMulti(int[] big, int[] small) {
9     int[][] nextElements = new int[small.length][big.length];
10    for (int i = 0; i < small.length; i++) {
11        nextElements[i] = getNextElement(big, small[i]);
12    }
13    return nextElements;
14 }
15
16 /* Do backwards sweep to get a list of the next occurrence of value from each
17  * index. */
18 int[] getNextElement(int[] bigArray, int value) {
19     int next = -1;
20     int[] nexts = new int[bigArray.length];
21     for (int i = bigArray.length - 1; i >= 0; i--) {
22         if (bigArray[i] == value) {
23             next = i;
24         }
25         nexts[i] = next;
26     }
27     return nexts;
28 }
29
30 /* Get closure for each index. */
```

```

31 int[] getClosures(int[][] nextElements) {
32     int[] maxNextElement = new int[nextElements[0].length];
33     for (int i = 0; i < nextElements[0].length; i++) {
34         maxNextElement[i] = getClosureForIndex(nextElements, i);
35     }
36     return maxNextElement;
37 }
38
39 /* Given an index and the table of next elements, find the closure for this index
40 * (which will be the min of this column). */
41 int getClosureForIndex(int[][] nextElements, int index) {
42     int max = -1;
43     for (int i = 0; i < nextElements.length; i++) {
44         if (nextElements[i][index] == -1) {
45             return -1;
46         }
47         max = Math.max(max, nextElements[i][index]);
48     }
49     return max;
50 }
51
52 /* Get shortest closure. */
53 Range getShortestClosure(int[] closures) {
54     int bestStart = -1;
55     int bestEnd = -1;
56     for (int i = 0; i < closures.length; i++) {
57         if (closures[i] == -1) {
58             break;
59         }
60         int current = closures[i] - i;
61         if (bestStart == -1 || current < bestEnd - bestStart) {
62             bestStart = i;
63             bestEnd = closures[i];
64         }
65     }
66     return new Range(bestStart, bestEnd);
67 }

```

This algorithm will potentially take $O(SB)$ time, where B is the length of `bigString` and S is the length of `smallString`. This is because we do S sweeps through the array to build up the next occurrences table and each sweep takes $O(B)$ time.

It uses $O(SB)$ space.

More Optimized

While our solution is fairly optimal, we can reduce the space usage. Remember the table we created:

value	7	5	9	0	2	1	3	5	7	9	1	1	5	8	8	9	7
index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
next 1	5	5	5	5	5	5	10	10	10	10	10	11	x	x	x	x	x
next 5	1	1	7	7	7	7	7	7	12	12	12	12	12	x	x	x	x
next 9	2	2	2	9	9	9	9	9	9	9	15	15	15	15	15	15	x
closure	5	5	7	9	9	9	10	10	12	12	15	15	x	x	x	x	x

In actuality, all we need is the closure row, which is the minimum of all the other rows. We don't need to store all the other next occurrence information the entire time.

Instead, as we do each sweep, we just update the closure row with the minimums. The rest of the algorithm works essentially the same way.

```
1  Range shortestSupersequence(int[] big, int[] small) {
2      int[] closures = getClosures(big, small);
3      return getShortestClosure(closures);
4  }
5
6  /* Get closure for each index. */
7  int[] getClosures(int[] big, int[] small) {
8      int[] closure = new int[big.length];
9      for (int i = 0; i < small.length; i++) {
10         sweepForClosure(big, closure, small[i]);
11     }
12     return closure;
13 }
14
15 /* Do backwards sweep and update the closures list with the next occurrence of
16  * value, if it's later than the current closure. */
17 void sweepForClosure(int[] big, int[] closures, int value) {
18     int next = -1;
19     for (int i = big.length - 1; i >= 0; i--) {
20         if (big[i] == value) {
21             next = i;
22         }
23         if ((next == -1 || closures[i] < next) &&
24             (closures[i] != -1)) {
25             closures[i] = next;
26         }
27     }
28 }
29
30 /* Get shortest closure. */
31 Range getShortestClosure(int[] closures) {
32     Range shortest = new Range(0, closures[0]);
33     for (int i = 1; i < closures.length; i++) {
34         if (closures[i] == -1) {
35             break;
36         }
37         Range range = new Range(i, closures[i]);
38         if (!shortest.shorterThan(range)) {
39             shortest = range;
40         }
41     }
42     return shortest;
43 }
```

This still runs in $O(SB)$ time, but it now only takes $O(B)$ additional memory.

Alternative & More Optimal Solution

There's a totally different way to approach it. Let's suppose we had a list of the occurrences of each element in `smallArray`.

value	7	5	9	9	2	1	3	5	7	9	1	1	5	8	8	9	7
index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

1 -> {5, 10, 11}
 5 -> {1, 7, 12}
 9 -> {2, 3, 9, 15}

What is the very first valid subsequence (which contains 1, 5, and 9)? We can just look at the heads of each list to tell us this. The minimum of the heads is the start of the range and the max of the heads is the end of the range. In this case, the first range is [1, 5]. This is currently our “best” subsequence.

How can we find the next one? Well, the next one will not include index 1, so let’s remove that from the list.

1 -> {5, 10, 11}
 5 -> {7, 12}
 9 -> {2, 3, 9, 15}

The next subsequence is [2, 7]. This is worse than the earlier best, so we can toss it.

Now, what’s the next subsequence? We can remove the min from earlier (2) and find out.

1 -> {5, 10, 11}
 5 -> {7, 12}
 9 -> {3, 9, 15}

The next subsequence is [3, 7], which is no better or worse than our current best.

We can continue down this path each time, repeating this process. We will end up iterating through all “minimal” subsequences that start from a given point.

1. Current subsequence is [min of heads, max of heads]. Compare to best subsequence and update if necessary.
2. Remove the minimum head.
3. Repeat.

This will give us an $O(SB)$ time complexity. This is because for each of B elements, we are doing a comparison to the S other list heads to find the minimum.

This is pretty good, but let’s see if we can make that minimum computation faster.

What we’re doing in these repeated minimum calls is taking a bunch of elements, finding and removing the minimum, adding in one more element, and then finding the minimum again.

We can make this faster by using a min-heap. First, put each of the heads in a min-heap. Remove the minimum. Look up the list that this minimum came from and add back the new head. Repeat.

To get the list that the minimum element came from, we’ll need to use a `HeapNode` class that stores both the `locationWithinList` (the index) and the `listId`. This way, when we remove the minimum, we can jump back to the correct list and add its new head to the heap.

```

1 Range shortestSupersequence(int[] array, int[] elements) {
2     ArrayList<Queue<Integer>> locations = getLocationsForElements(array, elements);
3     if (locations == null) return null;
4     return getShortestClosure(locations);
5 }
6
7 /* Get list of queues (linked lists) storing the indices at which each element in
8  * smallArray appears in bigArray. */
9 ArrayList<Queue<Integer>> getLocationsForElements(int[] big, int[] small) {
10     /* Initialize hash map from item value to locations. */
11     HashMap<Integer, Queue<Integer>> itemLocations =

```